```
'''
Content
  1. Random
  2. Time
  3. Datetime
  4. Function
  5. If-Else-Elif
  6. save dataframe into excel
'''
```

```
import sys
sys.version
```

```
'3.11.12 (main, Apr  9 2025, 08:55:54) [GCC 11.4.0]'
```

```
## Random number generation
'''
  1. Simulation based tasks - sensor data
  2. create randominzed values
'''
```

```
import random
```

```
## return float between 0 and 1
random.random()
```

```
0.6491008540621785
```

```
## return float between a and b

random.uniform(2,30)
```

```
23.406608167931928
```

```
## return an integer between a and b
```

```
random.randint(2,30)
```

```
27
```

```
## select one item randomly from a list

status = ["Normal","Alert","Maintenance"]
random.choice(status)
```

```
'Maintenance'
```

```
## TIP: To see the help over given code, write "?"

?random.choices
```

```
## select k randomly selected items instead of 1
random.choices(status,k=10)
```

```
['Normal',
 'Maintenance',
 'Normal',
 'Normal',
 'Maintenance',
 'Alert',
 'Normal',
 'Normal',
 'Alert',
 'Maintenance']
```

```
## give say more weightage to Normal
status = ["Normal","Alert","Maintenance"]
weights = [0.7,0.2,0.1]

random.choices(status,weights=weights,k=10)
```

```
['Normal',
 'Normal',
 'Normal',
 'Maintenance',
 'Normal',
```

```
      'Alert',
      'Normal',
      'Normal',
      'Alert',
      'Normal']
```

```
names = ["Aakash","Amit","Karthikeya","Shine"]
print(names)
```

```
['Aakash', 'Amit', 'Karthikeya', 'Shine']
```

```
## shuffling -- randomly reorder elements of a list
names = ["Aakash","Amit","Karthikeya","Shine","Sandeep","Chinmay",
random.shuffle(names)
print(names)
```

```
['Amit', 'Shine', 'Kiran', 'Sandeep', 'Aakash', 'Chinmay', 'Ka
```

```
print(random.uniform(2,30))
```

```
23.331191814056687
```

```
## fix the random behavior -- important for consistent simulation

random.seed(42)
print(random.uniform(2,30))
```

```
19.903950356820744
```

```
## Time, DateTime


import time
print(time.time()) ## returns the current time in seconds since th
```

```
1748767067.8538308
```

```
## pause execution for given number of seconds
print("Hi")
time.sleep(2)
print("How are you ?")
```

```
Hi
How are you ?
```

```
## convert seconds passed to the readable string
time.ctime(1748767067.8538308)
```

```
'Sun Jun  1 08:37:47 2025'
```

```
time.localtime()
```

```
time.struct_time(tm_year=2025, tm_mon=6, tm_mday=1,
tm_hour=8, tm_min=41, tm_sec=24, tm_wday=6, tm_yday=152,
tm_isdst=0)
```

```
## format the time

now = time.localtime()

print(time.strftime("%y-%m-%d %H:%M:%S", now))
print(time.strftime("%Y-%m-%d %H:%M:%S", now))
```

```
25-06-01 08:43:29
2025-06-01 08:43:29
```

```
from datetime import datetime
now = datetime.now()
print(now)
```

```
2025-06-01 08:44:39.765225
```

```
!pip install pytz
```

```
Requirement already satisfied: pytz in /usr/local/lib/python3
```

```
import pytz


india_timezone = pytz.timezone('Asia/Kolkata')
uk_timezone = pytz.timezone('Europe/London')
utc_now = datetime.now(pytz.utc)

india_time = utc_now.astimezone(india_timezone)
uk_time = utc_now.astimezone(uk_timezone)
print(f"Time in India: {india_time} and in UK: {uk_time}")
```

```
Time in India: 2025-06-01 14:18:59.667878+05:30 and in UK: 20
```

```
## add or subtract time

from datetime import timedelta
now = datetime.now()

future = now + timedelta(days=5, hours=2)

print(f"Current time: {now} and Future time: {future}")
```

```
Current time: 2025-06-01 08:52:24.606884 and Future time: 202
```

```
??dt
```

Start coding or generate with AI.

```
## parse string to Datetime

timestamp = "2025-06-01 18:45:00"
dt = datetime.strptime(timestamp, "%Y-%m-%d %H:%M:%S")

year = dt.year
month = dt.month
```

```
day = dt.day
hour = dt.hour
minute = dt.minute
second = dt.second
microsecond = dt.microsecond
tzinfo = dt.tzinfo
print(f''' Year: {year}, Month: {month}, Day: {day}, Hour: {hour}
Minute: {minute}, Second: {second}, TimezoneInfo: {tzinfo}''')
```

```
 Year: 2025, Month: 6, Day: 1, Hour: 18
Minute: 45, Second: 0, TimezoneInfo: None
```

```
## Function

'''
1. What ? -- Reusable block of code that performs specific task.
2. Why ? -- Reduce code repetition and improve readability. Write
 Easier to debug and maintain
 Modular design
3. Analogy -- Think of function like a machine tools that performs
4. Describe
  A. def -- keyword
  B. greet -- Function name
  C. () -- parenthesis
    C.1. name -
  D: : -- colon
5. Best Practices
  * Use meaningful function names..
  * keep one responsibility per function..
  * Write Comments/docstring..
'''


def greet(name):
  print(f"Hi {name}, Welcome to function concept !!")


greet(name = "Sumit")
```

```
Hi Sumit, Welcome to function concept !!
```

```
## Function with parameters


def show_temp(machine_name,temp):
  print(f"Machine {machine_name} has temperature {temp}")


show_temp("CNC",30.4)
```

```
Machine CNC has temperature 30.4
```

```
## Function which returns values


def calculate_efficiency(units_produced, units_defective):
  yield_units = units_produced - units_defective
  efficency = (yield_units/units_produced)*100
  return efficency


eff = calculate_efficiency(100,20)


print(eff)
```

```
80.0
```

```
## Conditional Logic

'''
1. To control the flow of program
2. if, elif, else
3. Syntax
  if condition:
    ## code if condition is True
  elif another_condition:
    ## code if this second condition is True
  else:
    ## Code if none of the above are True
'''
```

```
## Basic If
temperature = 45
if temperature>40:
  print("Alert: High Temperature!")
```

```
Alert: High Temperature!
```

```
## If-Else

temperature = 34
if temperature>40:
  print("Alert: High Temperature!")
else:
  print("Normal Temperature")
```

```
Normal Temperature
```

```
## If-Elif-Else

# temperature=34
temperature=20


if temperature>50:
  print("Very high temperature")
elif temperature>=30 and temperature<=50:
  print("High Temperature")
else:
  print("Normal Temperature")
```

```
Normal Temperature
```

```
## Multi condition logic with and, or
```

```
pressure = 95
temperature = 45

if pressure>90 and temperature>40:
  print("Warning: System Overload")
```

```
Warning: System Overload
```

```
if pressure>90 or temperature>40:
  print("System needs attention")
```

```
System needs attention
```

```
## Use in function


def check_status(temp,vibration):
  if temp>70:
    return "Shutdown due to overheat"
  elif vibration>8:
    return "Maintenance Required"
  else:
    return "Normal Operation"


check_status(72,7.5)
```

```
'Shutdown due to overheat'
```

```
### Nested If condition


def production_check(units, defects):
  if units>0:
    defect_rate = (defects/units) * 100
    if defect_rate > 5:
      return "High Defect Rate"
    else:
      return "Acceptable Quality"
```

```
  else:
    return "No Production"
```

```
production_check(0,7)
```

```
'No Production'
```

```
production_check(100,7)
```

```
'High Defect Rate'
```

```
## Compact If-Else (one line)
```

```
temp = 20
status = "Hot" if temp>25 else "Cold"
print(status)
```

```
Cold
```

```
## save dataframe into excel
```

```
import pandas as pd
units_produced = [random.randint(80,100) for _ in range(50)]
defective_units = [random.randint(0,10) for _ in range(50)]
data = {
    "Units Produced":units_produced,
    "Defective Units": defective_units
}

df = pd.DataFrame(data)
```

```
df.to_excel("data.xlsx")
```

```
df
```

	Units Produced	Defective Units
0	98	7
1	90	0
2	86	4
3	100	8
4	95	2
5	92	8
6	100	1
7	94	10
8	84	4
9	88	10
10	84	8
11	87	9
12	97	3
13	97	2
14	88	5
15	98	2
16	93	8
17	98	8
18	92	0
19	91	9
20	87	5
21	84	7
22	96	0
23	95	1

24	82	5
25	81	4
26	83	3
27	84	0
28	100	3
29	85	9
30	93	1
31	99	1
32	82	7
33	92	1
34	92	8
35	99	2
36	94	2